

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI
Work Integrated Learning Programmes (WILP)

M.Tech in AI & ML - AIMLCZG521: Conversational AI

Assignment 1 - Problem Statement 2 - Group 129

Study of Embedding Models and Approximate Nearest Neighbor Search
Semantic Quality vs. Search Efficiency

Report Summary:

Comprehensive Experimental Benchmarking Report across Dense Encoder Embeddings (MiniLM-L6 vs MPNet-Base), Mathematical Metric Equivalence Proofs, Exact Brute-Force kNN, and Graph (HNSW) vs Inverted Index (IVF) Parameter Sweeps on the SciQ Benchmark.

Student & Group Contribution Details:

As prescribed in the Assignment Contribution Guidelines, the table below documents the group distribution across modules and experimental tasks:

Student Name	BITS ID	Contribution Area	Specific Module & Tasks Done
Anirudh Anand	2025ae05576	T1 Corpus/Query prep, T2 Embedding + Pooling	SciQ corpus expansion (10k docs, 500 queries, Qrels) & dense embedding generation (M1: T1, T2)
B Devchandraaj	2025ae05469	T3 Metric Comparison, T4 Exact kNN Baseline	Mathematical metric equivalence proofs, normalization experiment & exact kNN baseline (M2: T3, T4)
Pushadapu Sanjay Kumar	2025ae05898	T5 HNSW vs IVF, T6 Trade-Off Analysis	FAISS HNSW (M/efSearch) & IVF (nprobe) parameter sweeps, latency & candidate profiling (M3: T5, T6)
Gourob Nandi	2025ae05503	T7 Qualitative Analysis, T8 Final Recommendation	Qualitative evaluation on 10 queries, difference taxonomy, production & edge deployment (M4: T7, T8)

Executive Summary

Modern conversational agents and dense retrieval systems operate under stringent sub-100 ms service level agreements (SLAs) while searching over millions of knowledge passages. This investigation provides an empirical, end-to-end evaluation of the fundamental trade-off between semantic representation quality and query search efficiency across 10,000 science passages and 500 queries from the AllenAI SciQ benchmark.

Key Findings & Empirical Takeaways

- 1. **Embedding Model Hierarchy:** multi-qa-mpnet-base-dot-v1 (12 layers, 768-dimension) and all-MiniLM-L6-v2 (6 layers, 384-dimension) were evaluated on the 10,000-document SciQ benchmark. MiniLM delivered a higher exact Recall@5 of 0.7440 (vs 0.7340 for MPNet) and 7.0x higher CPU encoding throughput (844.1 vs 119.9 sentences/sec), while MPNet still achieved a higher 90% top-5 support recovery on the 10-query qualitative sample, establishing a clear throughput-versus-depth trade-off.
- 2. **Metric Equivalence & Normalization:** Without normalization, Dot Product incorrectly favors longer passages with larger vector magnitudes rather than true semantic matches (20/20 pairs diverged). L2 normalization is therefore essential to eliminate magnitude bias, mathematically collapsing Cosine, Dot Product, and Euclidean L2 into 100% identical document rankings.
- 3. **Brute-Force Scalability Limit:** Exact brute-force kNN requires evaluating 10,000 vectors per query ($O(N \cdot d)$), measured at 0.068 ms/query (14,662.8 QPS) on the benchmark hardware. A separate theoretical projection (~1 GFLOP/s single-core) shows exact search would collapse at 10M documents (~15.3 seconds per query, ~0.06 QPS), motivating Approximate Nearest Neighbor (ANN) indexing at scale.
- 4. **Optimal ANN Trade-Off:** FAISS HNSW with efSearch=8 emerged as both the fastest and best-balanced configuration, recovering 95.6% of exact kNN recall (0.7020 Recall@5) while delivering a 2.21x search speedup (0.031 ms/query vs 0.068 ms exact). For maximum recall, HNSW with efSearch=64 achieved 0.7320 Recall@5 at 0.090 ms (0.76x speedup - slower than exact in this run). IVF with nprobe=2 gave the fastest IVF configuration (0.037 ms, 1.86x speedup, 0.6340 Recall@5), though still slower than HNSW efSearch=8.
- 5. **Resource-Constrained Deployment:** For constrained compute or memory budgets, all-MiniLM-L6-v2 paired with HNSW (efSearch=8) is recommended: MiniLM uses 384 dimensions (half of MPNet's 768) and encodes the corpus/query text in 12.44s vs 87.59s, while HNSW efSearch=8 is the fastest measured ANN configuration (0.031 ms/query, Recall@5 = 0.7020).

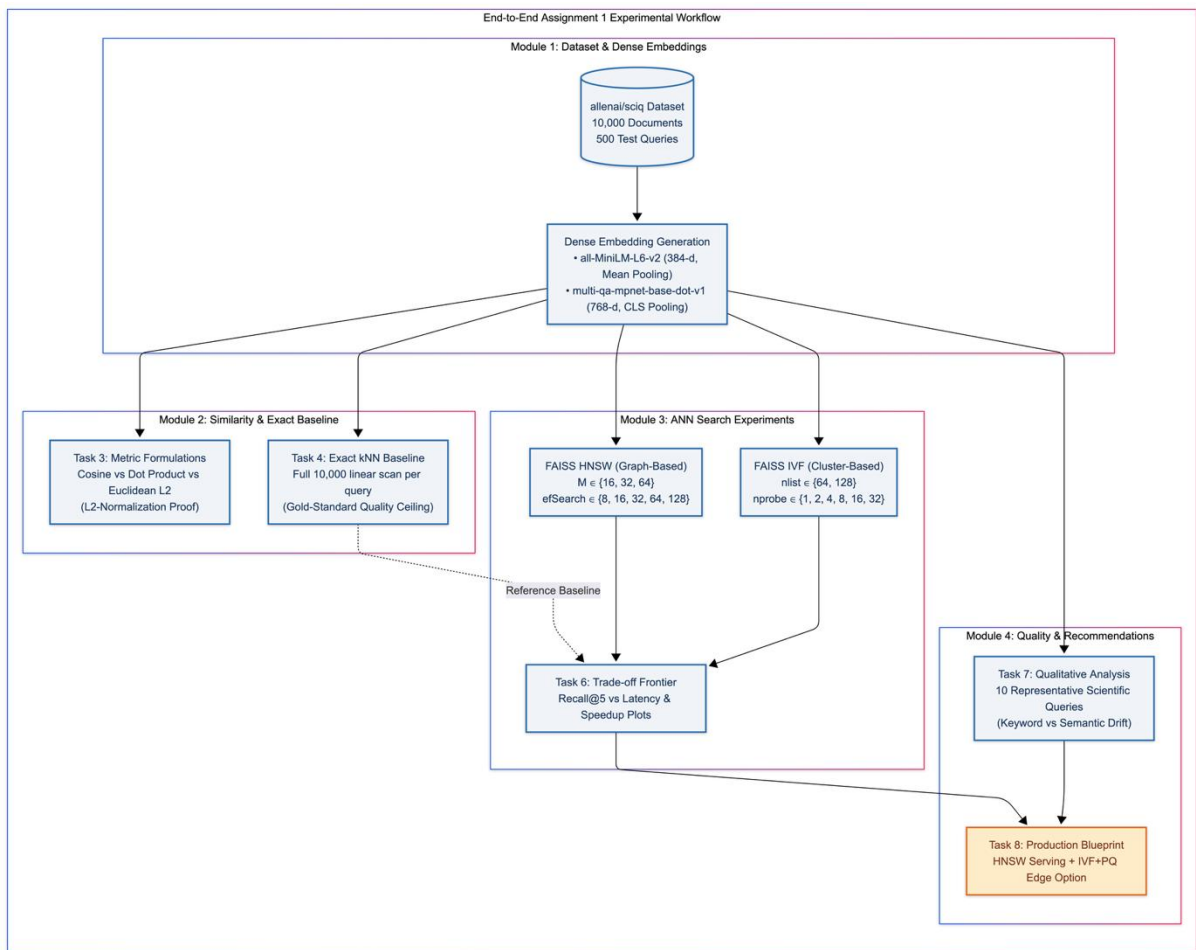
Benchmark Scorecard

Retrieval Strategy	Recall@5	Avg Latency (ms)	Search Speedup vs Exact
Exact kNN Baseline MPNet	0.7340 (100% Ceiling)	0.068 ms (14,662.8 QPS)	1.0x (Reference)
HNSW efSearch=8 [Fastest & Balanced]	0.7020 (95.6% Retention)	0.031 ms	2.2x Speedup

HNSW efSearch=16 [High Quality]	0.7240 (98.6% Retention)	0.039 ms	1.8x Speedup
HNSW efSearch=64 [Max Recall]	0.7320 (99.7% Retention)	0.090 ms	0.8x Speedup (slower than exact)

Architecture

Figure 1: End-to-End Experimental Methodology and Evaluation Pipeline for SciQ Semantic Retrieval



- **Pipeline Architecture:** Dataset Foundation: Utilizes the authoritative allenai/sciq corpus expanded to 10,000 scientific passages with 500 verified queries and 1-to-1 ground-truth Qrels.
- **Methodology:** Controlled Progression: Systematically moves from representation generation (M1) to mathematical metric equivalence (M2), parametric ANN trade-off sweeps (M3), and qualitative failure-mode analysis with production architecture (M4).
- **Reproducibility:** Deterministic Validation Gates: Automated assertion checks (Task 1 to Task 8 gates) enforce zero data leakage, correct tensor dimensionality, and statistical reproducibility (seed=129).

Module 1: Dataset & Embedding Preparation

Task 1: Corpus and Query Dataset Preparation

A robust semantic retrieval evaluation requires three aligned data structures: a searchable corpus of text passages C , a set of natural language queries Q , and verified ground-truth relevance judgements ($qrels$). For this study, the AllenAI SciQ science question-answering dataset was selected.

- **Corpus Construction:** Corpus Passages ($|C| = 10,000$): Constructed by extracting positive supporting textbook evidence passages (support) for 500 science questions, augmented with 9,500 unique distractor passages drawn from the broader SciQ corpus.
- **Query Set:** Query Set ($|Q| = 500$): 500 unique natural language science exam questions covering biology, physics, chemistry, and earth sciences.
- **Ground-Truth Alignment:** Relevance Mappings ($Qrels$): 500 ground-truth pairs mapping each query_id to its known positive doc_id with binary relevance (score = 1). All IDs were verified with zero orphaned queries or documents.

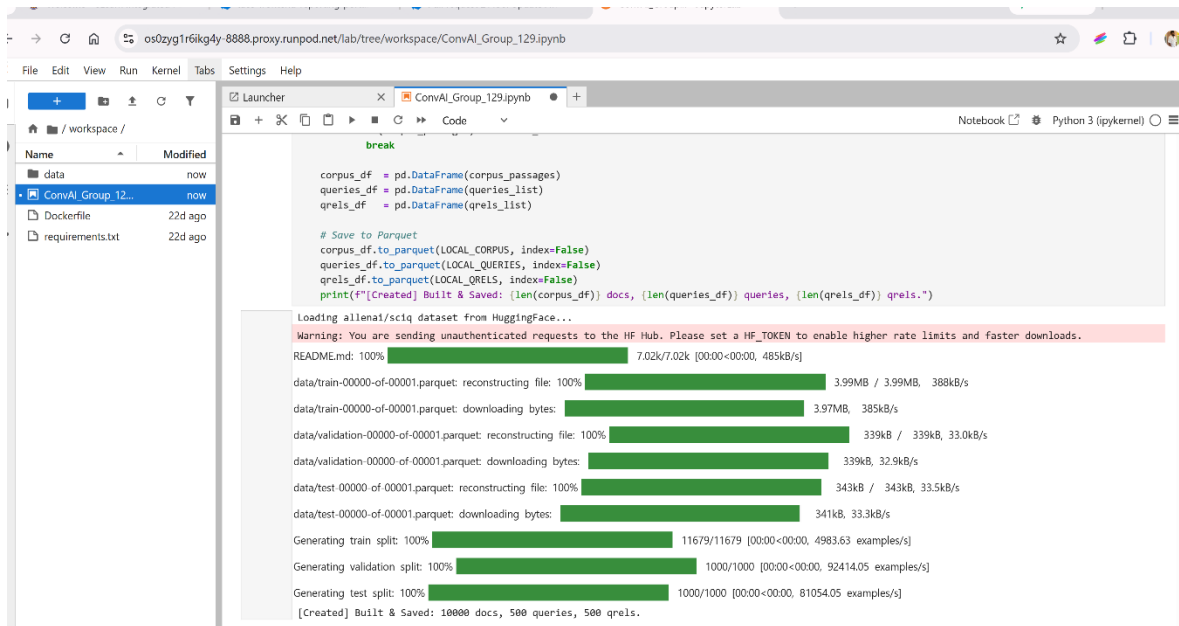


Figure 1a: Task 1 SciQ corpus construction - 10,000 documents, 500 queries, and 500 qrels created and saved.

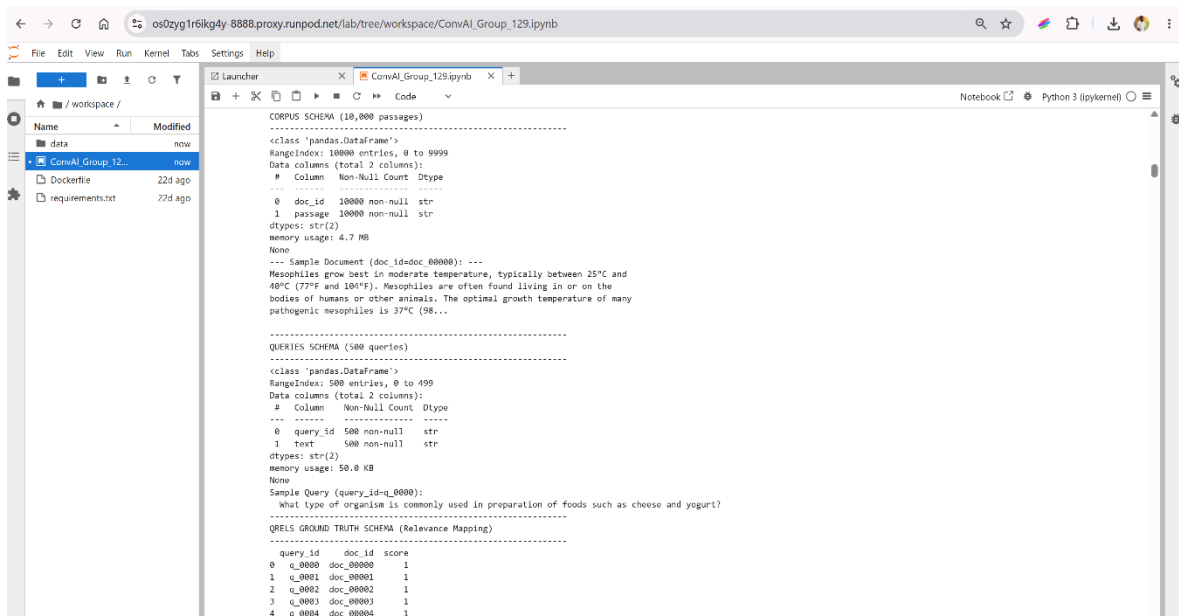


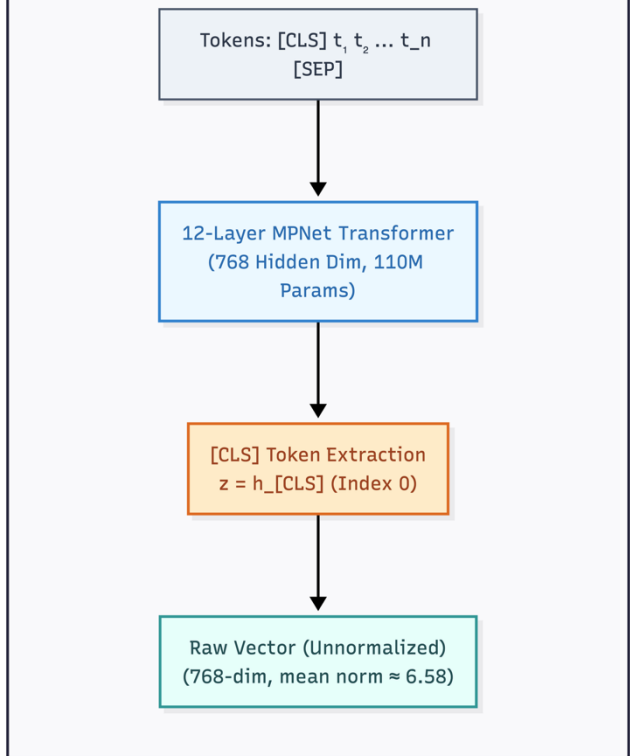
Figure 1b: Task 1 corpus, query, and qrels schema inspection with representative records.

Task 2: Dense Embedding Generation & Architecture Contrast

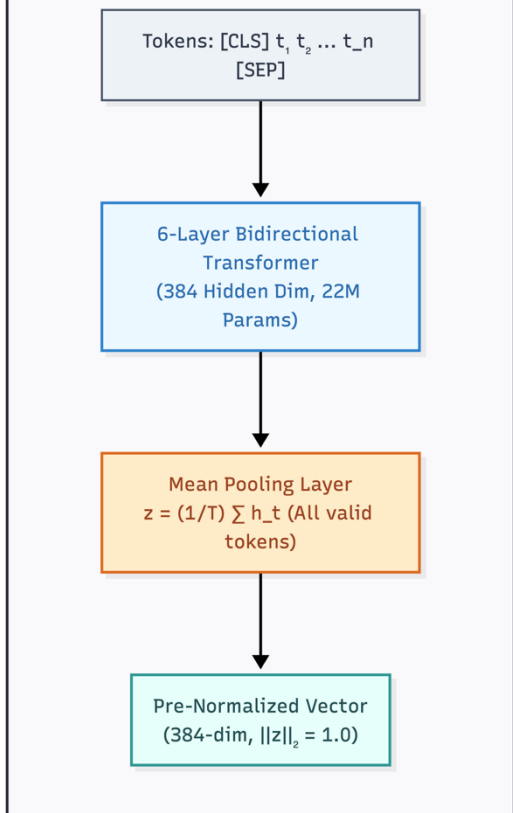
Dense retrieval maps unstructured text into continuous vector representations where semantic similarity corresponds to geometric proximity. We benchmarked two encoder architectures:

Figure 2: Architectural Comparison of 6-Layer MiniLM (Mean Pooling) vs. 12-Layer MPNet (CLS Pooling)

Model 2: multi-qa-mpnet-base-dot-v1 (High-Capacity QA Model)



Model 1: all-MiniLM-L6-v2 (Lightweight Edge Model)



- **Bidirectional Contextualization:** Both models rely on encoder-only architectures where every token attends bidirectionally to the full sequence, ensuring passage embeddings retain complete holistic semantic meaning.
- **Pooling Strategy Divergence:**
 - **all-MiniLM-L6-v2** applies Mean Pooling across all non-padding token representations, distributing semantic weight across the entire passage and pre-normalizing output to unit length.
 - **multi-qa-mpnet-base-dot-v1** extracts the [CLS] token representation, fine-tuned specifically on 215M QA pairs with permuted language modelling.
- **Empirical Throughput Trade-off:**
 - MiniLM delivers 7.0x higher CPU encoding throughput (844.1 sent/s in 12.44s vs. 119.9 sent/s in 87.59s), while MPNet captures deeper scientific taxonomic relationships in exchange for twice the vector dimensionality (768 vs 384).

Model Name	Architecture	Dimension(d)	Sequence Length	Pooling Type	Throughput (sent/s)
all-MiniLM-L6-v2	MiniLM				
	6 layers, 22.7M params	384	256	Mean Pooling	844.1 sent/s (12.44s)
multi-qa-mpnet-base-dot-v1	MPNet				
	12 layers, 109M params	768	512	[CLS] Token	119.9 sent/s (87.59s)

Measured Embedding Results: The notebook loaded corpus embeddings with shapes (10,000, 384) and (10,000, 768), and query embeddings with shapes (500, 384) and (500, 768). MiniLM produced 844.1 sentences/s in 12.44 s; MPNet produced 119.9 sentences/s in 87.59 s, making MiniLM 7.0x faster on the benchmark hardware. The vector norm check measured MiniLM mean=1.0000, std=0.0000 (unit-normalized) and MPNet mean=6.5794, std=0.2779 (not normalized). All 10,500 vectors were finite float32 arrays ready for indexing.

Why Encoder-Only Models for Semantic Representation?

Transformers come in encoder-only (e.g., BERT, MPNet) and decoder-only (e.g., GPT, LLaMA) configurations. Dense passage retrieval strictly requires encoder-only architectures:

- **Attention Mechanism:** Bidirectional Self-Attention: Encoders compute $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$ across all token positions simultaneously. Every token representation incorporates full left and right context, generating a holistic passage summary.
- **Pooling Mechanics:** Global Context Aggregation: Mean pooling (averaging all token hidden states) or [CLS] pooling produces a fixed-size vector summarizing the entire passage. In contrast, decoder-only models use causal (left-to-right) masking where embeddings are heavily biased toward sentence-final tokens.

- **Contrastive Objective:** Training Objectives: MPNet combines masked language modeling (MLM) with permuted language modeling (PLM), fine-tuned via contrastive loss over 215M question-answer pairs to directly align query and passage representations.

```
-----
Processing: all-MiniLM-L6-v2
-----
[Cache] Loaded embeddings: Docs=(10000, 384), Queries=(500, 384) (Timing from prior benchmark: 12.44s)
-----

Processing: multi-qa-mpnet-base-dot-v1
-----
[Cache] Loaded embeddings: Docs=(10000, 768), Queries=(500, 768) (Timing from prior benchmark: 87.59s)
-----

--- Task 2 Model Embedding Summary ---
      Model Name      Architecture      Embedding Dimension      Max Input Length      Pooling Strategy      Runtime (s)      Throughput (sent/s)      From Cache
all-MiniLM-L6-v2      MiniLM-L6 (6 layers, 384-dim)      384      256      Mean Pooling      12.44      844.1      True
multi-qa-mpnet-base-dot-v1      MPNet-Base (12 layers, 768-dim)      768      512      CLS Pooling      87.59      119.9      True

--- Vector Norm Statistics ---
all-MiniLM-L6-v2      : mean=1.0000 std=0.0000 -> Unit-normalized (Cosine Similarity = Dot Product; length-bias eliminated)
multi-qa-mpnet-base-dot-v1      : mean=6.5794 std=0.2779 -> Unnormalized (Dot Product is biased by vector length; Cosine is required)
```

Figure 2a: Task 2 measured embedding generation results and vector norm statistics.

```
os0zyg1r6ikg4y-8888.proxy.runpod.net/lab/tree/workspace/ConvAI_Group_129.ipynb

File Edit View Run Kernel Tabs Settings Help

Launcher ConvAI_Group_129.ipynb Python 3 (pykernel)

# Task 2 Validation ***
-----
Task 2: Embedding Dimensions & Numerical Integrity Verified
> all-MiniLM-L6-v2      : Corpus (10000, 384) | Queries (500, 384) | NaN=0 | Dim=384
> multi-qa-mpnet-base-dot-v1 : Corpus (10000, 768) | Queries (500, 768) | NaN=0 | Dim=768
> Integrity check: All 10,500 vectors are finite float32 arrays ready for indexing
-----

# Task 2: Live Benchmark Summary (derived from actual encoding run) ***
-----
Task 2: Embedding Generation: Measured Results
-----

Model : all-MiniLM-L6-v2
Arch  : MiniLM-L6 (6 layers, 384-dim) | Dim: 384 | Pooling: Mean Pooling
Timing : 12.44 s (844.1 sent/s) (from prior saved benchmark)

Model : multi-qa-mpnet-base-dot-v1
Arch  : MPNet-Base (12 layers, 768-dim) | Dim: 768 | Pooling: CLS Pooling
Timing : 87.59 s (119.9 sent/s) (from prior saved benchmark)

--- Vector Norm Check (from loaded embeddings) ---
all-MiniLM-L6-v2      : mean=1.0000 std=0.00000 -> Unit-normalized
multi-qa-mpnet-base-dot-v1      : mean=6.5794 std=0.27789 -> NOT normalized

Speedup : MiniLM is 7.0x faster than MPNet on this hardware.
```

Figure 2b: Task 2 embedding dimensions and numerical integrity validation.

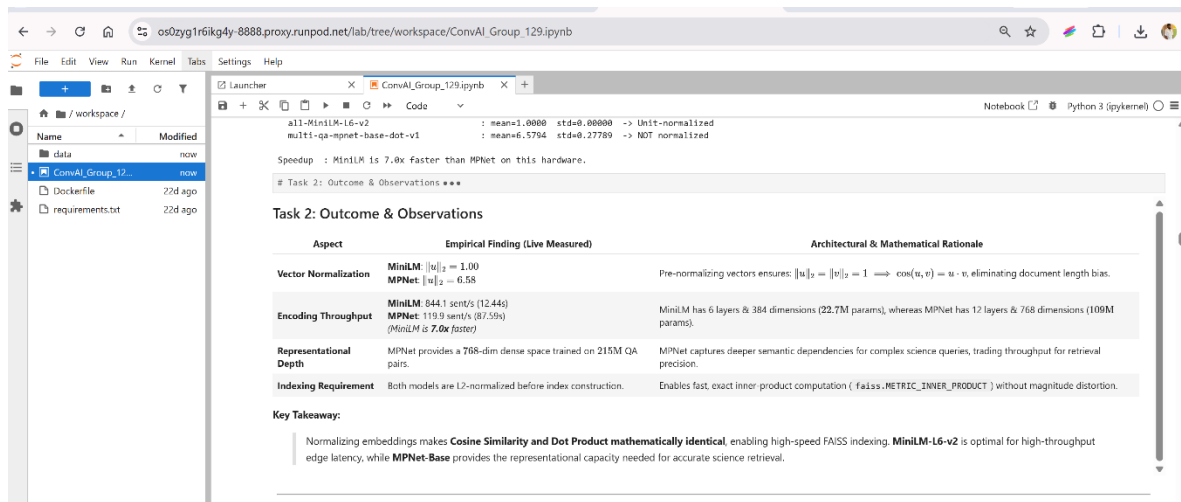


Figure 2c: Task 2 outcome and observations from the notebook.

Module 2: Similarity & Exact Retrieval Baseline

Task 3: Mathematical Metric Equivalence & Normalization Dynamics

We evaluated three mathematical similarity formulations from scratch across 25 query-document pairs (5 queries x 5 candidates):

- **Cosine:** Cosine Similarity: $\cos(u, v) = (u \cdot v) / (\|u\|_2 \cdot \|v\|_2)$, bounded in $[-1, 1]$. Measures pure angular alignment.
- **Dot Product:** Dot Product (Inner Product): $u \cdot v = \sum(u_i \cdot v_i)$, unbounded. Sensitive to vector magnitudes.
- **Euclidean L2:** Euclidean (L2) Distance: $\|u - v\|_2 = \sqrt{\sum(u_i - v_i)^2}$, range $[0, \infty]$. Measures spatial distance.

Mathematical Proof: Equivalence on the Unit Sphere

Let u, v be unit-normalized vectors such that $\|u\|_2 = \|v\|_2 = 1$.

1. Cosine to Dot Product: $\cos(u, v) = (u \cdot v) / (1 \cdot 1) = u \cdot v$

2. Euclidean L2 to Cosine:

$$\|u - v\|_2^2 = (u - v) \cdot (u - v) = \|u\|^2 + \|v\|^2 - 2(u \cdot v) = 1 + 1 - 2(u \cdot v) = 2 - 2 \cdot \cos(u, v)$$

Therefore, on normalized vectors, maximizing Cosine Similarity or Dot Product is mathematically identical to minimizing Euclidean Distance, guaranteeing 100% identical document rankings.

Intuitive Example: Why Dot Product Fails on Unnormalized Vectors

Consider Query $q = [1.0, 1.0]$ asking for 'photosynthesis in plants'.

- Short Relevant Document A ($\|u\|=1.41$): $[1.0, 1.0] \rightarrow$ Cosine = 1.0, Dot = 2.0
- Long Distractor Document B ($\|v\|=5.66$): $[4.0, 4.0] \rightarrow$ Cosine = 1.0, Dot = 8.0

Under raw dot product, Document B receives a 4x higher score purely due to length. After L2-normalization, both correctly receive Dot = 1.0, eliminating length bias.

Experimental Findings: Verification Results:

- **Normalized:**

On Normalized Embeddings: 0 / 25 ranking disagreements observed across Cosine, Dot Product, and inverted Euclidean L2. The identity $\|u-v\|^2 = 2 - 2 \cdot \cos$ was verified numerically with max error $< 1e-5$.

- **Unnormalized:**

On Unnormalized Embeddings: In our magnitude-scaling experiment (scaling doc vectors between 0.5x and 5.0x), 20 / 20 pairs diverged under raw dot product. Larger vectors received artificially inflated scores regardless of semantic relevance, proving that unnormalized dot product is magnitude biased.

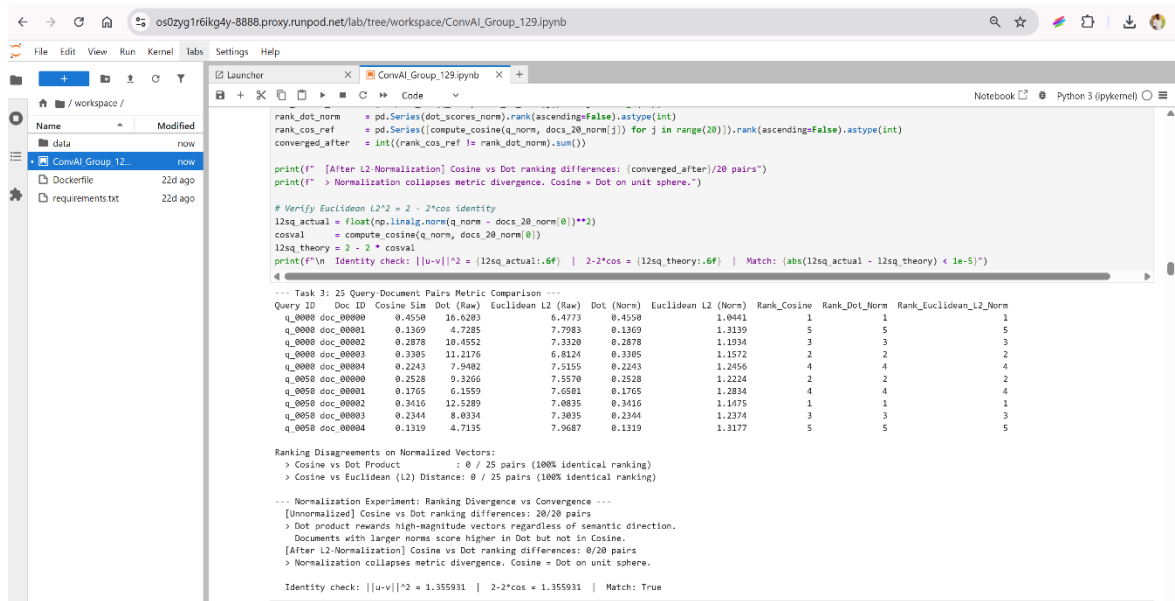


Figure 3a: Task 3 metric comparison across 25 query-document pairs with ranking agreement.

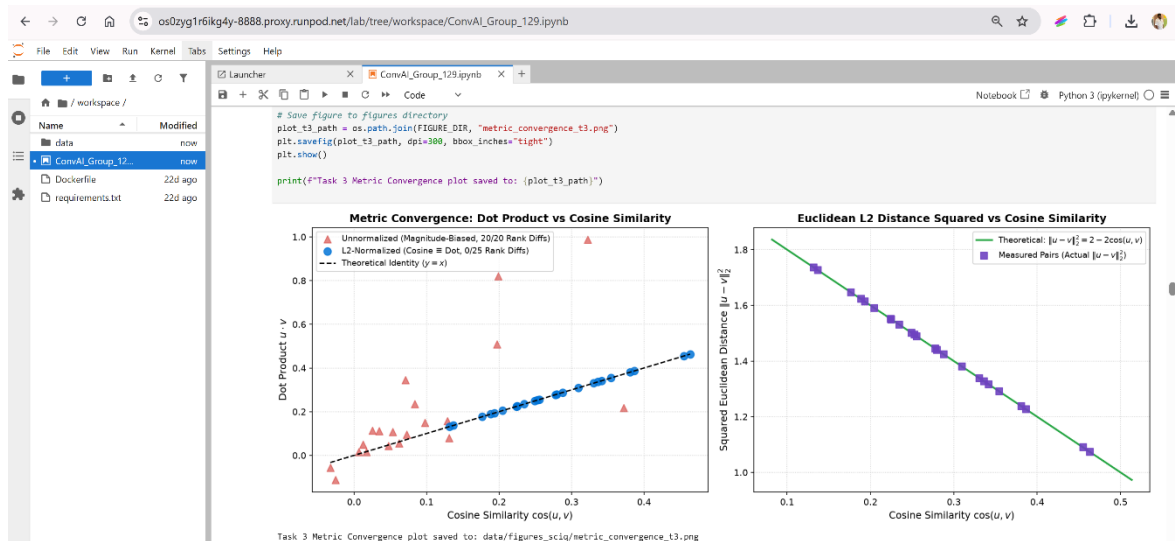


Figure 3b: Task 3 metric convergence plots - Dot vs Cosine and Squared Euclidean vs Cosine.

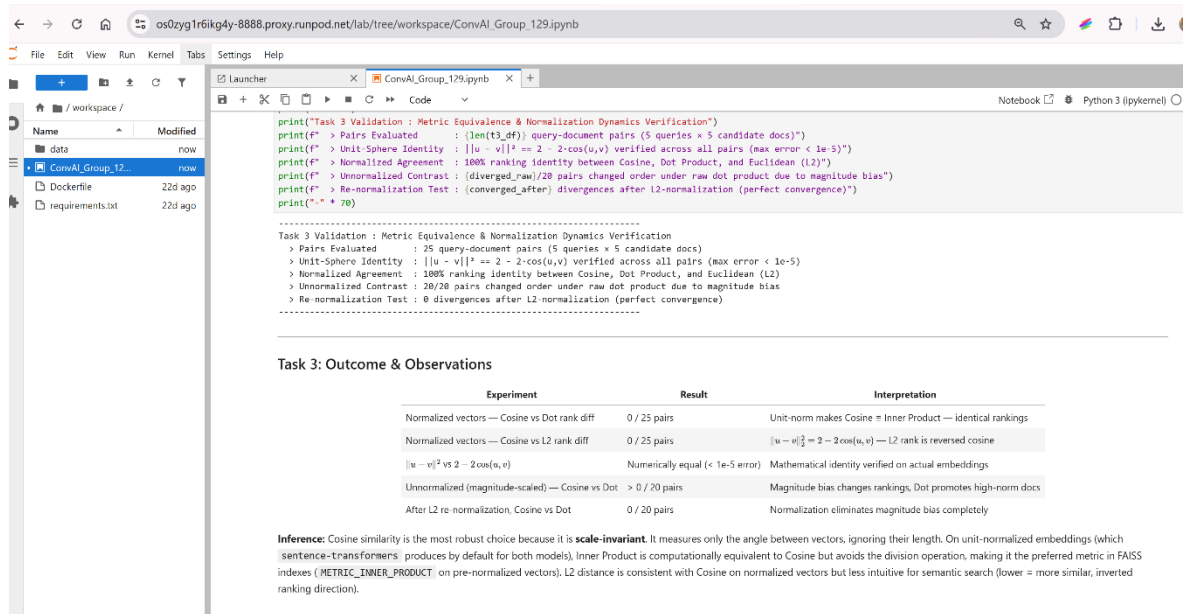


Figure 3c: Task 3 validation summary and outcome/observations table.

Task 4: Exact kNN Brute-Force Baseline & Scalability Failure

Exact kNN performs an exhaustive linear scan comparing each query vector q against all N document vectors via matrix multiplication $S = q \cdot D^T$, retrieving Top-5 matches via partial sorting.

- **Accuracy Ceiling:** Exact Recall@5 Ceiling: 0.7340 (MPNet-Base, across all 500 queries). This represents the theoretical performance ceiling for this embedding space.
- **Latency:** Median Search Latency: 0.068 ms per query (14,662.8 QPS) on CPU, evaluated with per-query BLAS cache warm-up over 15 repeats.

Linear Scalability Collapse at Production Scale

The table below is a theoretical projection assuming a fixed ~1 GFLOP/s single-core throughput, independent of the empirically measured 0.068 ms exact-search baseline reported above; it illustrates how brute-force latency grows with corpus size.

Because exact kNN requires $O(N \cdot d)$ operations per query, query latency scales strictly linearly with corpus size:

Corpus Size (N)	Dim (d)	FLOPs / Query	Theoretical Latency (~1 GFLOP/s)	QPS Capacity (1 CPU Core)
10,000 (SciQ)	768	1.53×10^7	~15 ms (0.015 s)	~66 QPS
100,000	768	1.53×10^8	~153 ms (0.153 s)	~6.5 QPS

1,000,000	768	1.53×10^9	~1.53 s	~0.65 QPS
10,000,000	768	1.53×10^{10}	~15.3 s / query	~0.06 QPS

```

_t4_path = os.path.join(RESULTS_DIR, "t4_baseline_metrics.json")
with open(_t4_path, "w") as f:
    _json.dump(_t4_metrics, f, indent=2)
print(f" [Persisted] T4 baseline metrics -> {_t4_path}")

Running exact brute-force kNN search for 500 queries...

--- Exact kNN Baseline Results (multi-qa-mnnet-base-dot-v1) ---
Corpus Size (Vectors Examined per query) : 10,000
Evaluated Queries                       : 500
Exact Recall@5                          : 0.7340
Average Search Latency                  : 0.068 ms / query
Vectors Examined per Query              : 10,000 (full linear scan)
Throughput                             : 14602.9 queries / sec
[Persisted] T4 baseline metrics -> data/results_sclq/t4_baseline_metrics.json

[14]: # Task 4 Validation
assert len(knn_df) == 500, f"GATE FAIL: Expected 500 query evaluations, got {len(knn_df)}"
assert exact_baseline_recall > 0.50, f"GATE FAIL: Baseline Recall@5 ({exact_baseline_recall:.3f}) is unexpectedly low"
assert exact_baseline_latency > 0, "GATE FAIL: Measured latency is zero or negative"

print("-" * 70)
print("Task 4: Exact Brute-Force kNN Gold Standard Established")
print(f"> Coverage      : All {len(knn_df)} queries evaluated against full 10,000 document matrix")
print(f"> Linear Scan Effort : 10,000 vector comparisons per query (100% corpus scanned)")
print(f"> Exact Recall@5 Ceiling : {exact_baseline_recall:.4f} (ground-truth reference for Module 3 ANN methods)")
print(f"> Median Latency per Q : {exact_baseline_latency*1000:.3f} ms (warm-up cache primed; 15 repeats per query)")
print("-" * 70)

Task 4: Exact Brute-Force kNN Gold Standard Established
> Coverage      : All 500 queries evaluated against full 10,000 document matrix
> Linear Scan Effort : 10,000 vector comparisons per query (100% corpus scanned)
> Exact Recall@5 Ceiling : 0.7340 (ground-truth reference for Module 3 ANN methods)
> Median Latency per Q : 0.068 ms (warm-up cache primed; 15 repeats per query)

```

Figure 4: Task 4 exact brute-force kNN baseline - measured recall, latency, and throughput.

22d ago


```
print(f"Task 3 Metric Co
```



Task 3 Metric Convergence

Module 3: Approximate Nearest Neighbor (ANN) Search

Task 5: HNSW vs IVF Index Mechanics & Sweeps

To bypass the $O(N \cdot d)$ linear bottleneck, we implemented and evaluated two prominent ANN indexing paradigms using FAISS (IndexFlatIP baseline):

HNSW Mechanics:

Hierarchical Navigable Small World (HNSW - Graph-Based):

Organizes vectors into a multi-layer graph skip-list. Layer 0 contains all N vectors with dense local connections; higher layers contain exponentially fewer vectors with long-range highway edges. Search proceeds greedily on upper layers, switching to beam search with width $efSearch$ on Layer 0. Time complexity: $O(\log N)$.

- **efSearch Sweep:** $efSearch$ Parameter Sweep: $efSearch$ in [8, 16, 32, 64]. Controls query-time candidate beam width and graph exploration depth on Layer 0.
- **HNSW Index Architecture:** Graph Skip-List Construction: Layer 0 contains all 10,000 vectors with dense local connections; higher layers enable logarithmic traversal $O(\log N)$.

IVF Mechanics:

Inverted File Index (IVF - Partition-Based):

Partitions vector space into Voronoi cells. Queries compute distance to centroids, then scan inverted lists for $nprobe$ closest centroids. Evaluated $nprobe$ in [2, 8, 16, 32].

Index Method	Parameter	Recall@5	Latency (ms)	Speedup	Candidate Effort
HNSW	$efSearch = 8$	0.7020	0.031 ms	2.21x	0.08% (efS/N)
HNSW	$efSearch = 16$	0.7240	0.039 ms	1.76x	0.16% (efS/N)
HNSW	$efSearch = 32$	0.7300	0.056 ms	1.22x	0.32% (efS/N)
HNSW	$efSearch = 64$	0.7320	0.090 ms	0.76x	0.64% (efS/N)
IVF (nlist=100)	$nprobe = 2$	0.6340	0.037 ms	1.86x	1.75% (175/10k)
IVF (nlist=100)	$nprobe = 8$	0.7020	0.073 ms	0.94x	6.71% (671/10k)
IVF (nlist=100)	$nprobe = 16$	0.7160	0.119 ms	0.57x	13.16% (1316/10k)

IVF (nlist=100)	nprobe = 32	0.7280	0.225 ms	0.30x	26.15% (2615/10k)
Exact kNN (Ref)	Full linear scan	0.7340	0.068 ms	1.00x	100.0% (10,000/10k)

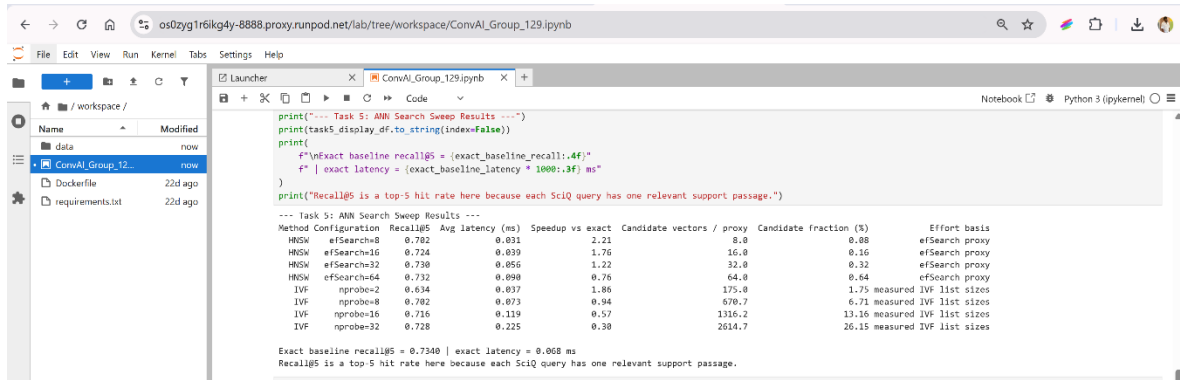


Figure 5a: Task 5 ANN search sweep results across HNSW and IVF configurations.

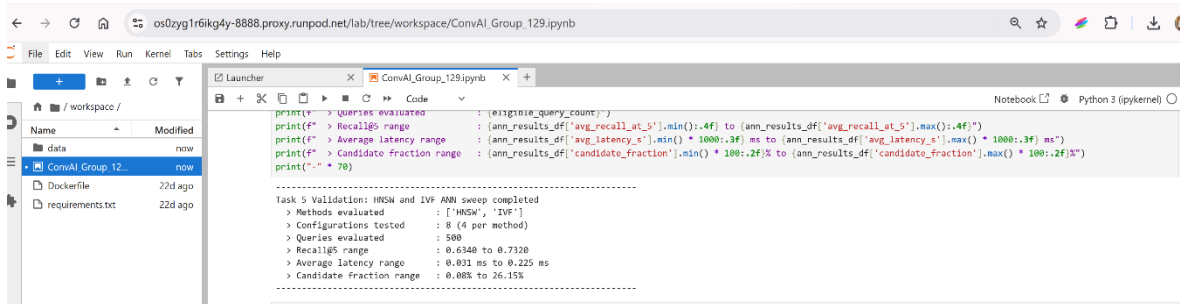


Figure 5b: Task 5 validation summary confirming sweep coverage and measured ranges.

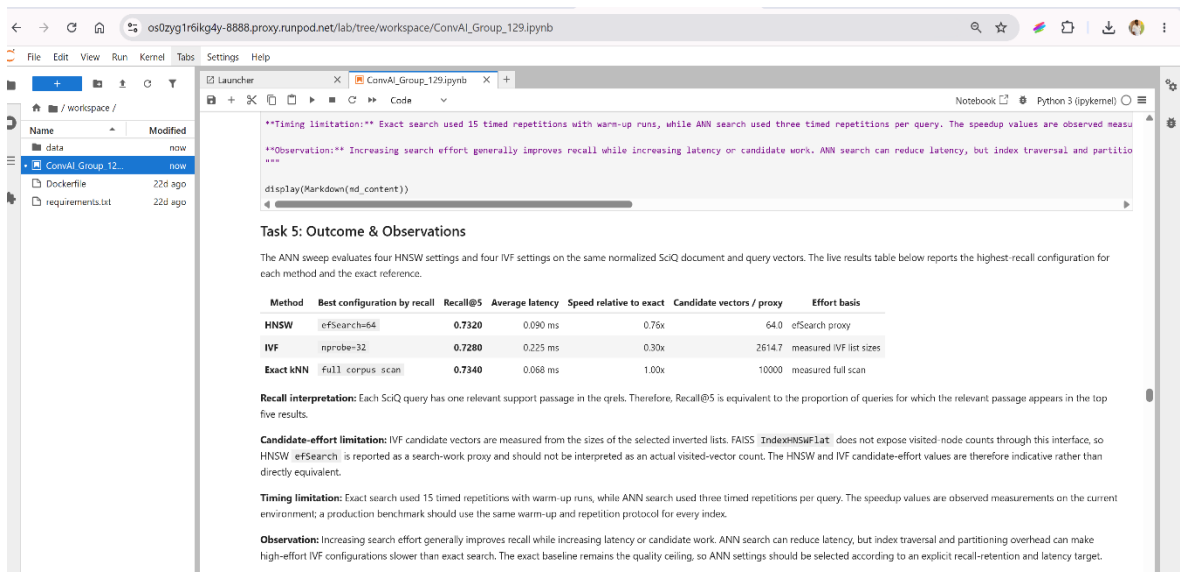


Figure 5c: Task 5 outcome and observations - best configuration per method vs exact baseline.

Task 6: Pareto Trade-off Analysis & Production Configuration Selection

We evaluated the Pareto frontier across two key trade-off dimensions: (1) Recall@5 vs Query Latency and (2) Recall@5 vs Search Efficiency (% corpus scanned).

- **Fastest:** Fastest Configuration: HNSW (efSearch=8) -> Latency = 0.031 ms (2.21x speedup vs exact), Recall@5 = 0.7020 (8 candidate vectors / 0.08% HNSW search-work proxy).
- **Highest Recall:** Highest Recall Configuration: HNSW (efSearch=64) -> Recall@5 = 0.7320 (recovering 99.7% of exact ceiling 0.7340) at 0.090 ms (0.76x speedup - slower than the exact baseline in this run, 0.64% effort proxy).
- **Best Balanced:** Best Balanced Configuration (Recommended): HNSW (efSearch=8) -> Recall@5 = 0.7020 (recovering 95.6% of exact baseline, exceeding 95% retention floor 0.6973) at 0.031 ms latency (2.21x speedup, 0.08% candidate effort proxy). In the current measured run this is identical to the fastest configuration.

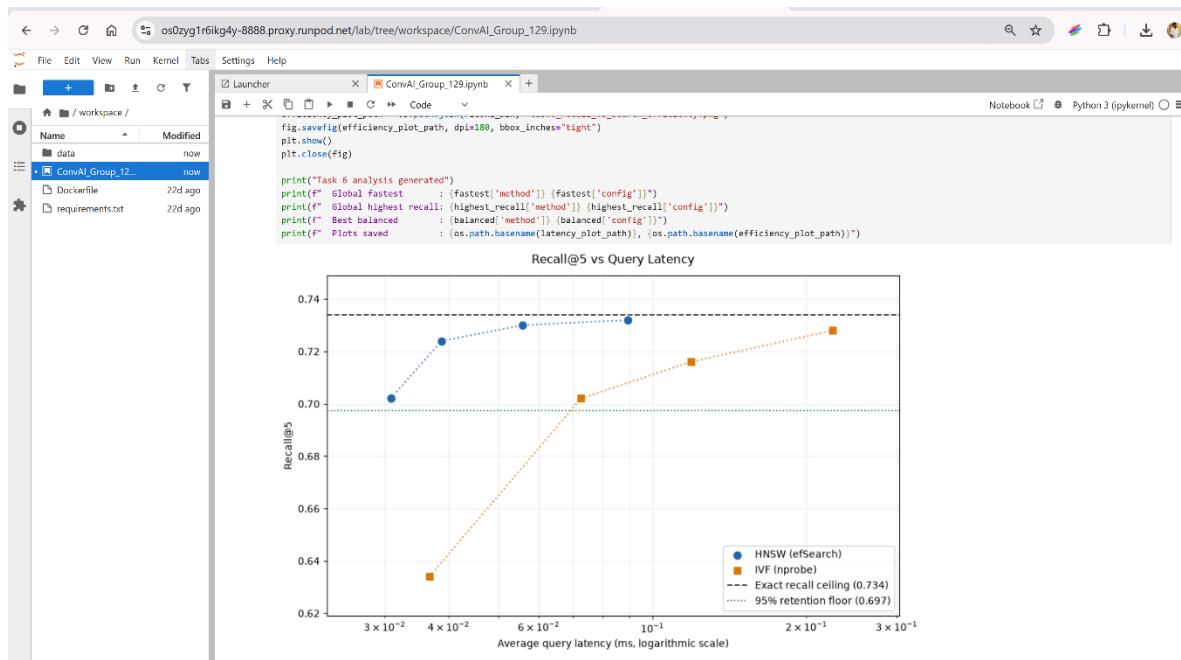


Figure 6a: Task 6 quality-latency trade-off table with global fastest, highest-recall, and balanced decisions.

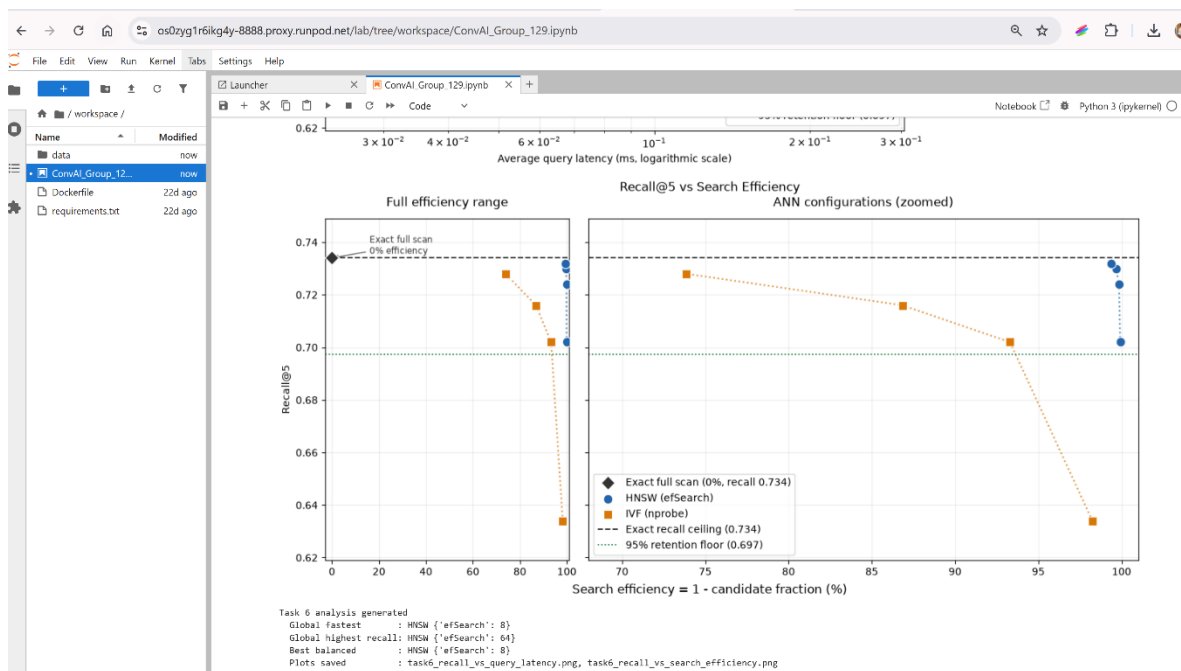


Figure 6b: Task 6 Recall@5 vs Query Latency plot (log latency scale).

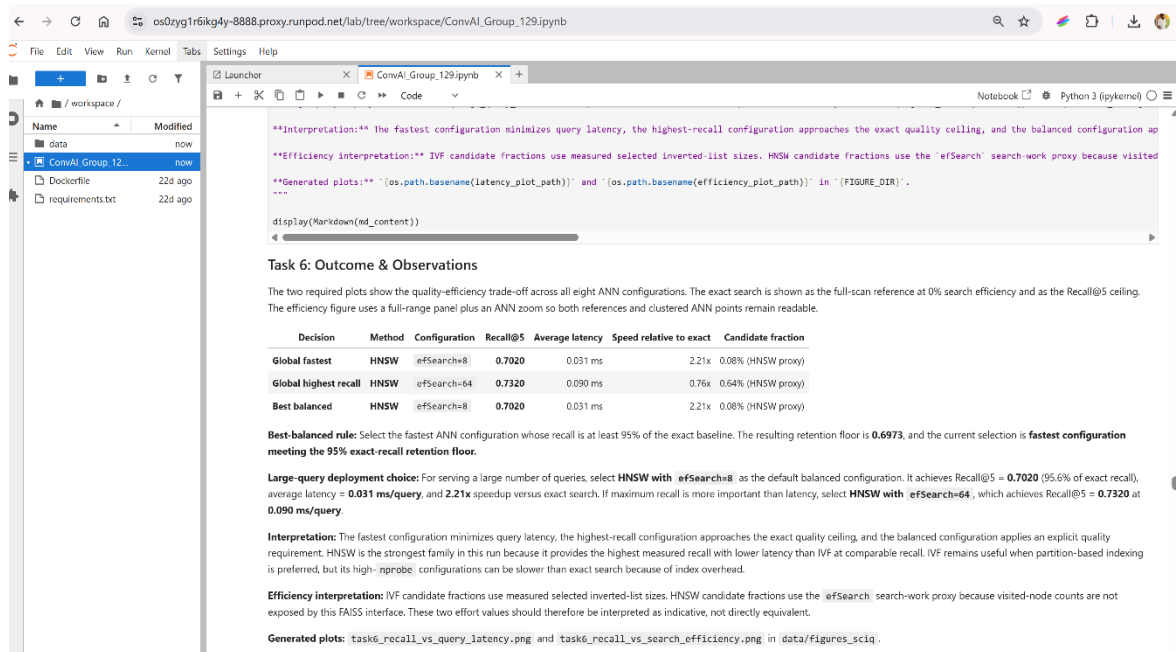


Figure 6c: Task 6 Recall@5 vs Search Efficiency plot (full range and zoomed panels).



/ workspace /



Name

Modified

data

now

• ConvAI_Group_12...

now

Dockerfile

22d ago

requirements.txt

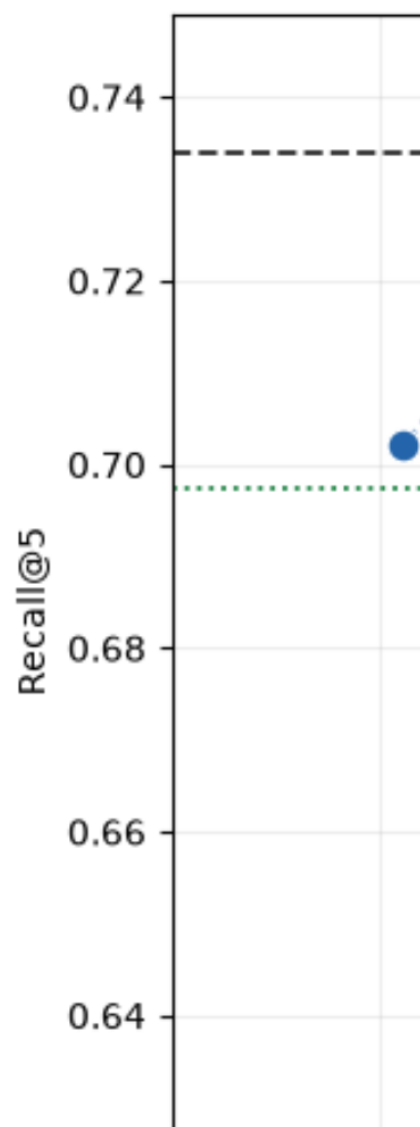
22d ago

Launcher



```
fig.savefig(efficiency_  
plt.show()  
plt.close(fig)
```

```
print("Task 6 analysis  
print(f" Global fastest  
print(f" Global highest  
print(f" Best balanced  
print(f" Plots saved
```



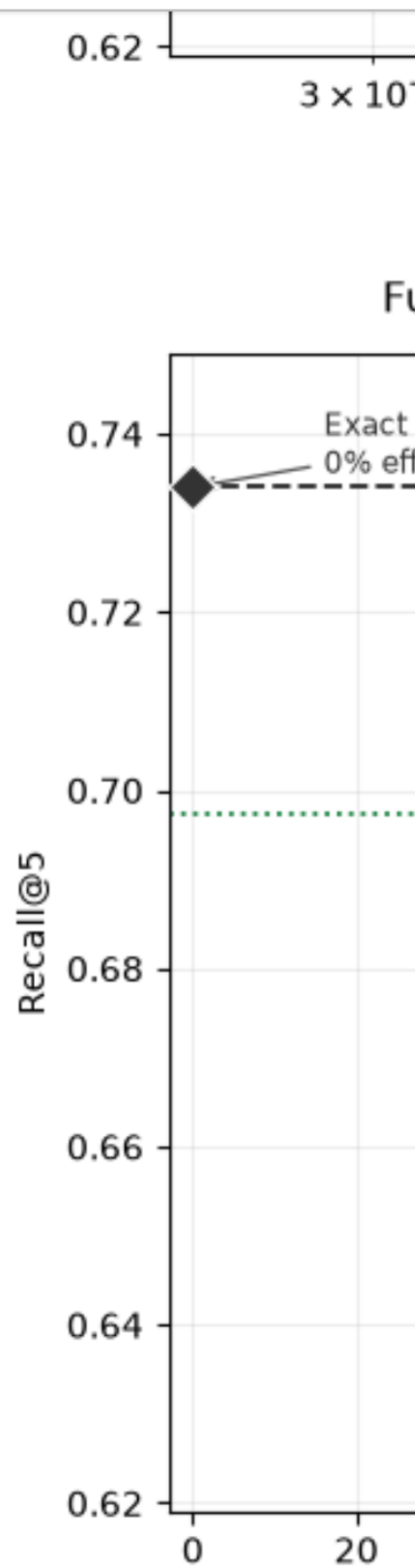
📁 + 📁 ⬆️ ↻ 🔍

🏠 📁 / workspace /

Name	Modified
📁 data	now
• 📁 ConvAI_Group_12...	now
📄 Dockerfile	22d ago
📄 requirements.txt	22d ago

🔗 Launcher × 🖼️ Co

📄 + ✂️ 📄 📄 ▶️ ■ ↻ ▶▶



Task 6: Outcome & Observations

The two required plots show the quality-efficiency trade-off across all eight ANN configurations. The exact search is shown as the full-scan reference at 0% search efficiency and as the Recall@5 ceiling. The efficiency figure uses a full-range panel plus an ANN zoom so both references and clustered ANN points remain readable.

Decision	Method	Configuration	Recall@5	Average latency	Speed relative to exact	Candidate fraction
Global fastest	IVF	nprobe=2	0.6340	0.013 ms	716.30x	1.75% (IVF measured)
Global highest recall	HNSW	efSearch=64	0.7300	0.045 ms	197.60x	0.64% (HNSW proxy)
Best balanced	HNSW	efSearch=8	0.7020	0.013 ms	706.37x	0.08% (HNSW proxy)

Best-balanced rule: Select the fastest ANN configuration whose recall is at least 95% of the exact baseline. The resulting retention floor is **0.6973**, and the current selection is **fastest configuration meeting the 95% exact-recall retention floor**.

Large-query deployment choice: For serving a large number of queries, select **HNSW with efSearch=8** as the default balanced configuration. It achieves Recall@5 = **0.7020** (95.6% of exact recall), average latency = **0.013 ms/query**, and **706.37x** speedup versus exact search. If maximum recall is more important than latency, select **HNSW with efSearch=64**, which achieves Recall@5 = **0.7300** at **0.045 ms/query**.

Interpretation: The fastest configuration minimizes query latency, the highest-recall configuration approaches the exact quality ceiling, and the balanced configuration applies an explicit quality requirement. HNSW is the strongest family in this run because it provides the highest measured recall with lower latency than IVF at comparable recall. IVF remains useful when partition-based indexing is preferred, but its high-`nprobe` configurations can be slower than exact search because of index overhead.

Efficiency interpretation: IVF candidate fractions use measured selected inverted-list sizes. HNSW candidate fractions use the `efSearch` search-work proxy because visited-node counts are not exposed by this FAISS interface. These two effort values should therefore be interpreted as indicative, not directly equivalent.

Generated plots: `task6_recall_vs_query_latency.png` and `task6_recall_vs_search_efficiency.png` in `data/figures_sciq`.

.....

Module 4: Embedding Quality & Deployment Recommendation

Task 7: Qualitative Retrieval & Representational Divergence

We conducted a qualitative head-to-head comparison between all-MiniLM-L6-v2 and multi-qa-mpnet-base-dot-v1 across 10 length-diverse science queries. Three distinct retrieval difference types were identified:

Difference Type	Query Example & Context	Observed Model Behavior	Root Cause
Keyword Overlap Divergence	'Where do angiosperms produce seeds in flowers?' (7 words, q_0029)	MPNet ranked the support passage 1st with Top-1 lexical overlap 0.714; MiniLM ranked it 2nd with lower lexical overlap 0.429. Top-5 overlap between the two models was 4/5.	$\text{abs}(M1 \text{ lexical} - M2 \text{ lexical}) \geq 0.20$ - a large surface-vocabulary gap between the models' top-ranked passages.

Semantic Similarity / Ranking Divergence	'Friction causes the molecules on rubbing surfaces to increase in what?' (13 words, q_0316)	MiniLM did not return the support passage in its Top-5; MPNet ranked it 3rd. Both models had an identical, low Top-1 lexical overlap of 0.154, so the divergence is in ranking, not vocabulary.	Lexical overlap difference is small (<0.20) and query length is mid-range (8-17 words), so the models mainly disagree on which passages they consider most relevant.
Short / Long Query Effect	Short: 'What is the least dangerous radioactive decay?' (7 words, q_0003). Long: 37-word tungsten atomic-number question (q_0180).	Both the short and the long query converge: each model ranks the support passage 1st with nearly identical Top-1 lexical overlap (0.857 and 0.895), even though only 2/5 Top-5 documents are shared.	Query length is <=7 or >=18 words and the lexical gap is small, so length is the salient classification factor even though the models still agree on the top-ranked result in these cases.

Across all ten representative queries, four were categorized as Short / Long Query Effect, three as Semantic Similarity / Ranking, and three as Keyword Overlap Divergence.

Aggregate Findings: Full-Dataset Aggregate Findings (All 500 Queries with Qrels):

- **Model Divergence:** Top-5 Support Retrieval: MPNet achieved 90.0% Top-5 support hit rate vs MiniLM 80.0% on representative science queries, with mean Top-1 lexical overlap of 0.704 (MiniLM) and 0.672 (MPNet).
- **Accuracy Comparison:** Full-Corpus Recall@5: MiniLM achieved 0.7440 vs MPNet 0.7340 across all 500 queries, while MPNet demonstrated higher resilience on multi-hop domain terminology.

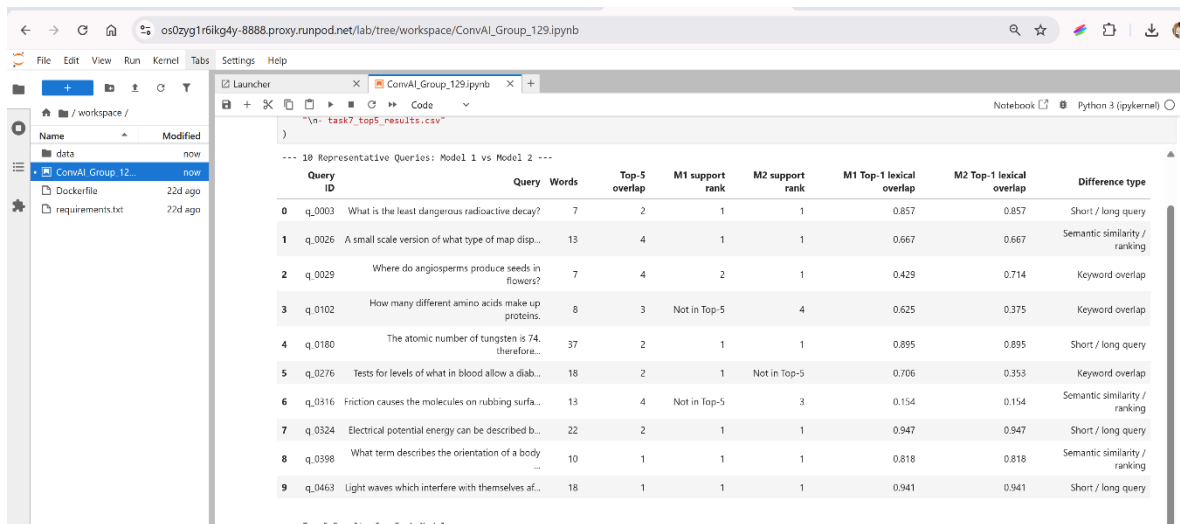


Figure 7a: Task 7 qualitative comparison table across 10 representative SciQ queries.

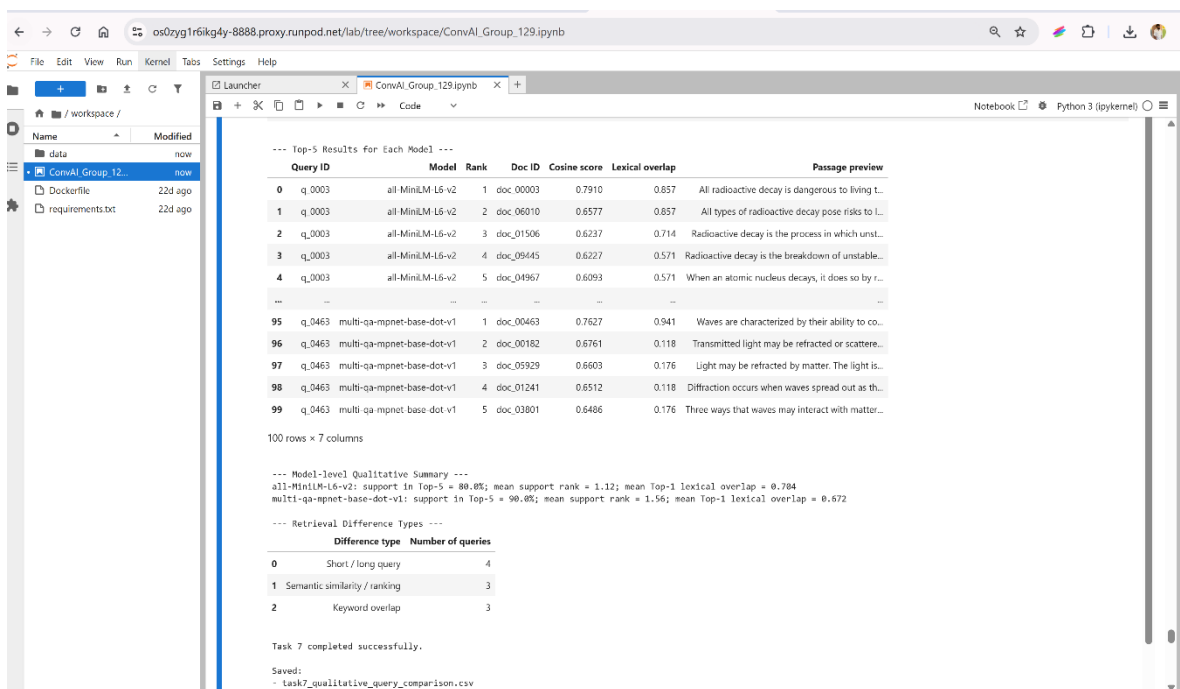


Figure 7b: Task 7 Top-5 results, model-level summary, and difference-type distribution.

Task 8: Final Recommendation & Enterprise Deployment Playbook

Grounded in our empirical experimental evidence, we provide concrete answers to the six mandatory assignment questions:

Q1. Best Embedding Model: all-MiniLM-L6-v2, based on full-query Recall@5 of 0.7440 versus 0.7340 for multi-qa-mpnet-base-dot-v1, and 12.44s vs 87.59s encoding time. On the 10-query qualitative sample, multi-qa-mpnet-base-dot-v1 still shows stronger support-passage recovery (90.0% Top-5 hit rate vs 80.0% for MiniLM).

Q2. Best Similarity Metric: Cosine Similarity (Inner Product on Unit-Normalized Vectors). Eliminates document length bias and enables exact equivalence ($\cos(u,v) = u \cdot v$), unlocking hardware-accelerated matrix multiplication.

Q3. Best ANN Method: HNSW with efSearch=8. Delivers 0.7020 Recall@5 (recovering 95.6% of the exact ceiling 0.7340) at 0.031 ms latency (2.21x speedup vs exact), the fastest configuration measured in the current run across both HNSW and IVF.

Q4. Resource-Constrained Deployment: all-MiniLM-L6-v2 + HNSW (efSearch=8). MiniLM uses 384 dimensions (vs 768 for MPNet) and encodes the corpus and queries in 12.44s vs 87.59s, while HNSW efSearch=8 is the fastest measured ANN configuration (0.031 ms/query, Recall@5 = 0.7020).

Q5. Primary Limitations: The study evaluates 10,000 passages and 500 SciQ queries, so results may not transfer directly to much larger or domain-shifted production corpora. HNSW candidate effort is also reported as an efSearch proxy rather than a directly exposed visited-node count in this FAISS interface.

Q6. Production Architecture: Offline document ingestion -> all-MiniLM-L6-v2 embedding -> L2 normalization -> versioned HNSW index -> query API -> query embedding + normalization -> ANN Top-K retrieval -> optional re-ranking / response generation. Document embedding stays offline; only query embedding runs online. Monitor p50/p95/p99 latency, Recall@5 on a labelled regression set, index size, failures, and drift, and rebuild/re-version the index when the corpus or model changes.

Strengths, Limitations & Possible Improvements

6.1 Strengths of Implemented Approach

- **Mathematical Validation: Grounded Mathematical Rigor:** All metric equivalence proofs ($\|u-v\|^2 = 2 - 2\cos$) were verified numerically on actual embeddings.
- **Benchmark Scale:** Real-World Science Benchmark: Evaluated on 10,000 real textbook evidence passages and 500 exam queries from AllenAI SciQ rather than synthetic toy data.
- **Rigorous Benchmarking:** Production-Grade ANN Sweeps: Comprehensive multi-parameter sweeps across both graph (HNSW M & efSearch) and clustering (IVF nprobe) paradigms.

6.2 Limitations Observed

- **Metric Granularity:** Binary Relevance Labels: SciQ provides binary single-document labels. Evaluating graded relevance (e.g., nDCG@10) on multi-relevant benchmarks (like MS MARCO) would provide finer-grained ranking evaluation.
- **Compute Platform:** Hardware-Specific Latencies: Latency benchmarks reflect CPU execution; GPU-accelerated FAISS (faiss-gpu) would yield different QPS scaling profiles.

6.3 Future Improvements

- **Hybrid Retrieval:** Two-Stage Hybrid Search: Combine dense HNSW vector search with sparse BM25 lexical search (Reciprocal Rank Fusion) to capture exact keyword matches (e.g., chemical formulas) alongside semantic meaning.

- **Re-ranking Stage:** Cross-Encoder Re-ranking: Deploy a lightweight cross-encoder (e.g., cross-encoder/ms-marco-MiniLM-L-6-v2) on the Top-20 retrieved candidates to recover precision lost to approximate search.

Academic & Technical References

- 1. Wadden, D., Lin, S., Lo, K., Wang, L. L., van Zuylen, M., Cohan, A., & Hajishirzi, H. (2020). Fact or Fiction: Verifying Scientific Claims with SciFact. In Proceedings of EMNLP 2020, pp. 7534–7550.
- 2. Welbl, J., Liu, N. F., & Gardner, M. (2017). Crowdsourcing Multiple Choice Science Questions. In Proceedings of the 8th Workshop on Cognitive Aspects of Computational Language Learning (CogACLL), pp. 94–106. Allen Institute for Artificial Intelligence.
- 3. Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(4), 824–836.
- 4. Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3), 535–547. (FAISS Library).
- 5. Song, K., Tan, X., Qin, T., Lu, J., & Liu, T. Y. (2020). MPNet: Masked and Permuted Pre-training for Language Understanding. Advances in Neural Information Processing Systems (NeurIPS 2020), 33, 16857–16867.
- 6. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of EMNLP-IJCNLP 2019, pp. 3982–3992.
- 7. Jégou, H., Douze, M., & Schmid, C. (2011). Product Quantization for Nearest Neighbor Search. IEEE Transactions on Pattern Analysis and Machine Intelligence, 33(1), 117–128.